1.2. Capacidades y mecanismos de ejecución de código de los navegadores Web (Criterio 1.2)

- **Resultado de Aprendizaje Asociado: RA1** (*Selecciona las arquitecturas y tecnologías de programación sobre clientes Web, identificando y analizando las capacidades y características de cada una*).

- **Criterio Curricular Oficial: CE 1.b** (*Se han identificado las capacidades y mecanismos de ejecución de código de los navegadores web.*).

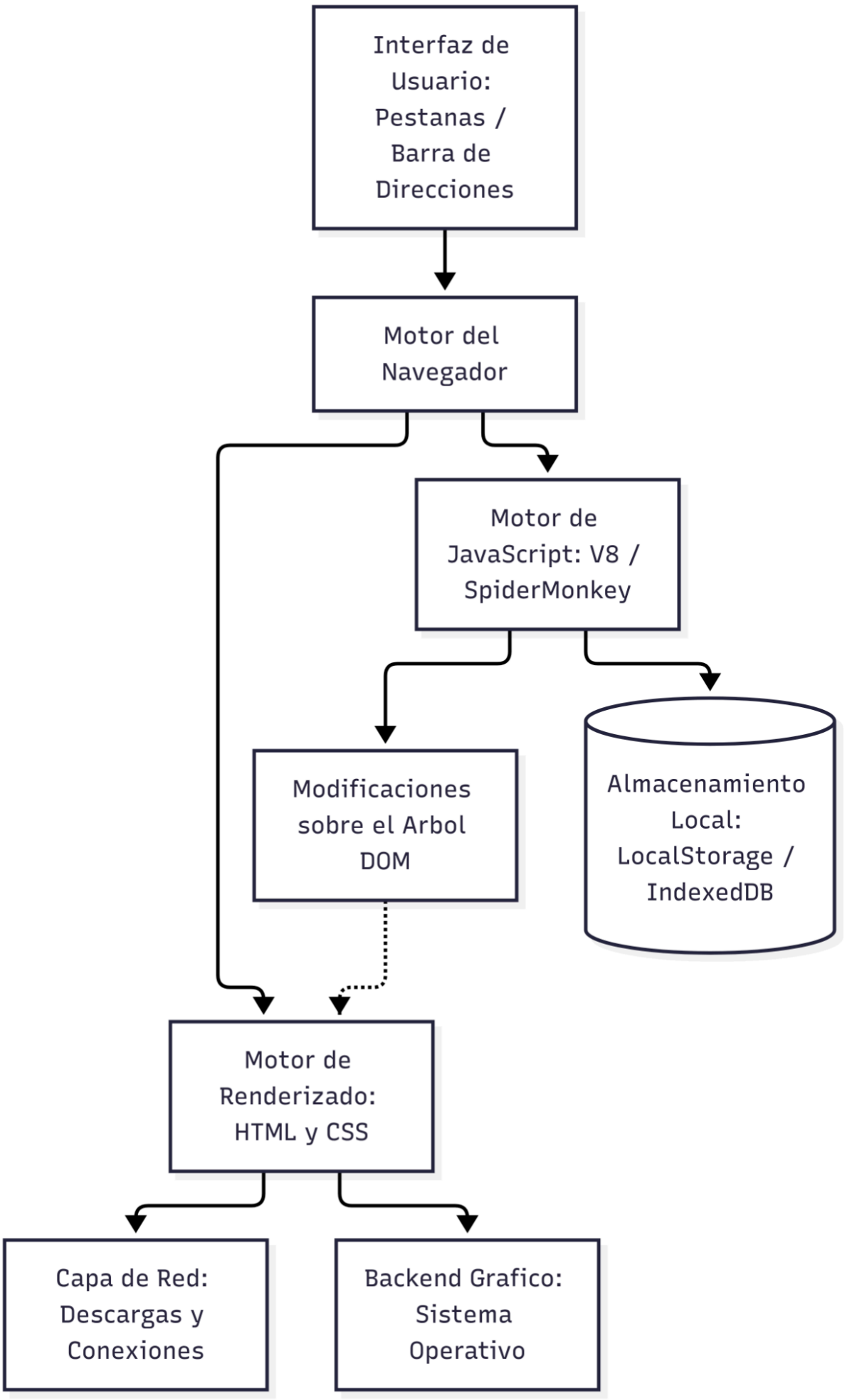
- **Ponderación del Criterio:** (**16,67% del RA1** | **0,833% sobre la calificación final del módulo**).

A. ¿Qué es un Navegador Web y Cómo se Organiza por Dentro?

Un navegador web es una aplicación cliente instalada en el ordenador o móvil del usuario cuyo trabajo principal consiste en pedir páginas a servidores web a través de la red (mediante protocolos como HTTP o HTTPS), interpretar el código que recibe (HTML, CSS y JavaScript) y mostrarlo en la pantalla de forma comprensible e interactiva.

Para que todo funcione sin fallos ni bloqueos, el interior de un navegador está dividido en siete partes o módulos de trabajo:

1. **Interfaz de usuario (*User Interface*):** Es la parte externa de la ventana con la que interactuamos directamente: la barra para escribir direcciones web, las pestañas, los botones de avanzar y retroceder, el botón de recargar y el menú de configuración.
2. **Motor del navegador (*Browser Engine*):** Hace de puente o intermediario entre la interfaz externa y los motores internos, gestionando órdenes generales como abrir una pestaña nueva o mostrar una ventana de alerta.
3. **Motor de renderizado (*Rendering Engine*):** Lee el documento HTML y las hojas de estilo CSS, calcula el tamaño y la posición exacta de cada elemento y los dibuja en la pantalla.
4. **Motor de JavaScript (*JavaScript Engine*):** Es el intérprete que lee las líneas de código JavaScript, las traduce a instrucciones que la máquina comprende y las ejecuta en el procesador. Emplea compilación **JIT (*Just-In-Time*)**, un mecanismo que detecta qué funciones se ejecutan muchas veces y las traduce directamente a código máquina nativo del procesador para ganar velocidad.
5. **Capa de red (*Networking*):** Se encarga de enviar y recibir datos a través de Internet (descargar imágenes, código, resolver nombres de dominio DNS y comprobar certificados de seguridad HTTPS).
6. **Backend de interfaz (*UI Backend*):** Conecta el navegador con el sistema operativo (Windows, Linux, macOS o Android) para dibujar cuadros de texto, ventanas o barras de desplazamiento con el aspecto nativo del sistema.
7. **Almacenamiento de datos (*Data Storage*):** Espacio en el disco duro del usuario donde el navegador guarda cookies, archivos en caché y bases de datos locales para que las páginas recuerden información.


Interfaz de Usuario: Pestanas / Barra de Direcciones
Motor del Navegador
Motor de JavaScript: V8 / SpiderMonkey
Modificaciones sobre el Arbol DOM
Almacenamiento Local: LocalStorage / IndexedDB
Motor de Renderizado: HTML y CSS
Capa de Red: Descargas y Conexiones
Backend Grafico: Sistema Operativo

Fundamental: Motores Separados

El motor que **dibuja** la página (motor de renderizado) y el motor que **ejecuta el código** (motor de JavaScript) son dos piezas distintas. Cuando en JavaScript escribimos una instrucción para cambiar un texto de la pantalla, el motor de JavaScript debe enviarle un aviso al motor de renderizado para que este vuelva a calcular y pintar ese trozo de la página.

B. Los Grandes Motores de Navegadores en la Actualidad

En el mercado conviven varios navegadores, pero la mayoría comparten los mismos motores internos:

Navegador habitual	Motor de Renderizado (Dibuja)	Motor de JavaScript (Ejecuta código)	Empresa u Organización
Google Chrome	**Blink**	**V8**	Google
Microsoft Edge	**Blink**	**V8**	Microsoft
Mozilla Firefox	**Gecko**	**SpiderMonkey**	Fundación Mozilla
Apple Safari	**WebKit**	**JavaScriptCore**	Apple
Brave / Opera	**Blink**	**V8**	Varios (Brave Software, Opera)

Toma Nota: Los Navegadores en iPhone y iPad

En los dispositivos móviles de Apple, las normas de su tienda de aplicaciones obligan a que cualquier navegador (aunque se llame Chrome o Firefox) use por dentro el motor **WebKit** de Safari.

Sabías que...: El Origen de Blink

En sus comienzos, Google Chrome utilizaba el motor WebKit de Apple. En el año 2013, Google decidió hacer una copia del proyecto y continuar su desarrollo por separado bajo el

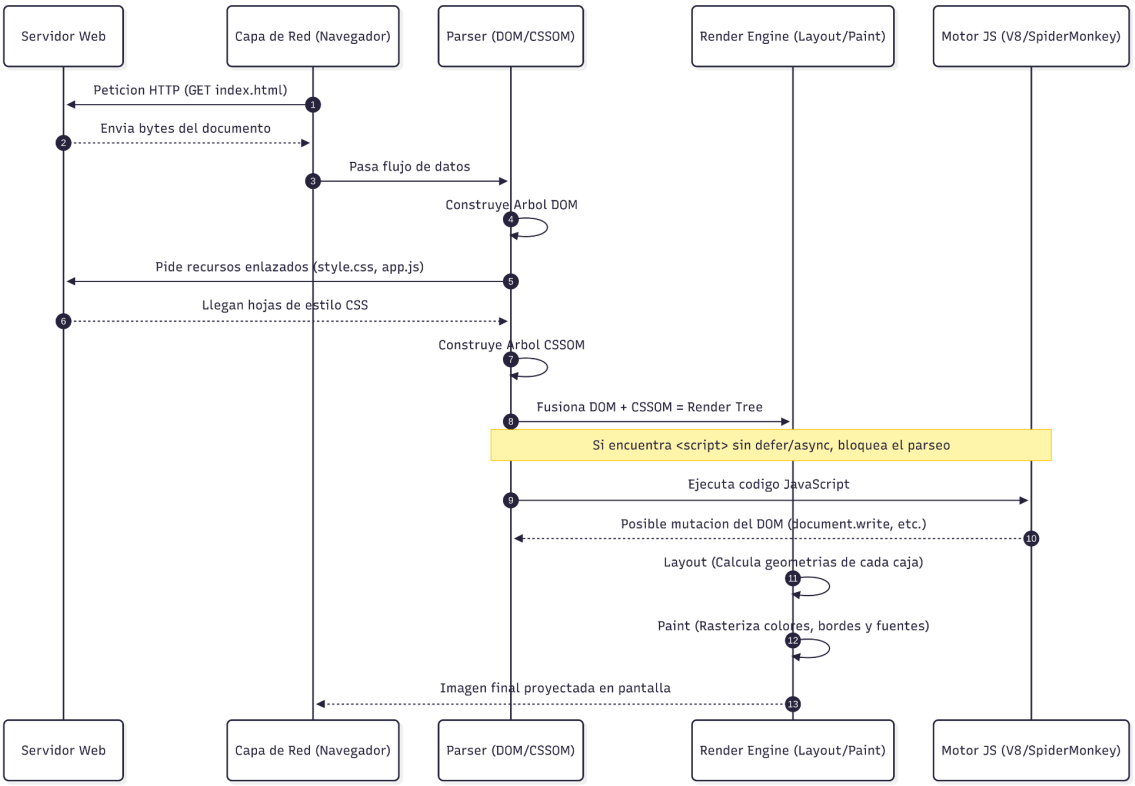
nombre de **Blink**, que es el motor que hoy usan la mayoría de navegadores basados en Chromium.

C. Cómo Transforma el Navegador el Código en Píxeles (El Proceso de Renderizado)

Cuando el navegador recibe el archivo HTML a través de la red, realiza cuatro pasos consecutivos para mostrarlo:

1. **Creación del DOM y CSSOM:** Lee el código HTML y construye en la memoria un árbol con todas las etiquetas (árbol DOM). Al mismo tiempo, lee el archivo CSS y genera un árbol con todas las reglas de color y estilo (árbol CSSOM).
2. **Unión en el Árbol de Renderizado (*Render Tree*):** Combina el árbol de etiquetas con el árbol de estilos. Aquí solo entran los elementos que realmente se van a ver. Las etiquetas que sirven solo para configuración (como <head>) o los elementos que tengan la regla CSS display: none; quedan fuera de este árbol porque no ocupan espacio visual.
3. **Disposición (*Layout*):** El navegador calcula el ancho, el alto y las coordenadas exactas de cada caja en la pantalla según el tamaño de la ventana.
4. **Pintado (*Paint*):** Dibuja los colores, bordes, tipografías e imágenes píxel a píxel sobre la pantalla.

Cuidado con los scripts: Si el navegador encuentra una etiqueta <script> mientras lee el HTML, detiene la lectura hasta que el archivo JavaScript se descarga y se ejecuta por completo. Si el script es muy pesado, la pantalla se quedará en blanco durante unos instantes.



D. Capacidades Nativas del Navegador (APIs Web)

El navegador moderno proporciona funciones ya preparadas a las que podemos acceder directamente desde JavaScript:

1. **Manipulación de la página (DOM):** Modificar textos, cambiar colores, ocultar cajas o crear nuevos elementos cuando el usuario pulsa un botón.
2. **Peticiones en segundo plano:** Mediante la instrucción fetch(), el navegador puede pedir datos a un servidor web y mostrarlos sin tener que recargar la página entera.
3. **Guardar datos en el equipo del usuario:**
    - **Cookies:** Textos muy pequeños (hasta 4 KB) que se envían al servidor en cada petición, útiles para mantener la sesión iniciada.
    - **sessionStorage:** Guarda datos mientras la pestaña siga abierta. Si se cierra la pestaña, la información se borra.
    - **localStorage:** Guarda datos de forma permanente en el equipo (hasta 5 o 10 MB). Aunque el usuario apague el ordenador, los datos siguen ahí.
    - **IndexedDB:** Una pequeña base de datos dentro del navegador para guardar grandes volúmenes de información cuando la aplicación necesita funcionar sin conexión a Internet.
4. **Acceso a dispositivos físicos (siempre con permiso del usuario):**
    - Conocer la ubicación geográfica (GPS o antenas Wi-Fi) con la API de Geolocalización.
    - Utilizar la cámara y el micrófono para videollamadas.
    - Consultar si el equipo tiene batería suficiente o si está conectado a la red.

Para Saber Más: Comprobar la Compatibilidad

No todos los navegadores incorporan las novedades al mismo tiempo. Por eso, en JavaScript se suele comprobar si una función existe antes de usarla:

JavaScript

```
if ('geolocation' in navigator) {

  // El navegador soporta geolocalización

} else {

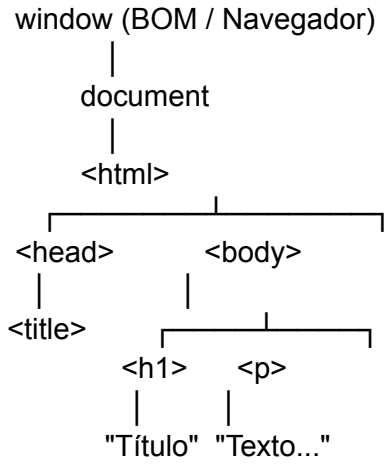
  // El navegador es antiguo y no la soporta

}
```

Además, los programadores consultan el sitio web Can I Use para ver en qué versiones de cada navegador funciona cada característica.

E. El DOM (*Document Object Model*) como Puente de Comunicación

El navegador toma el documento HTML descargado y construye en la memoria RAM una estructura en forma de árbol invertido llamada **DOM** (*Document Object Model*).



- **Definición técnica:** El DOM es la representación estructurada de todos los elementos que forman la página web (enlaces, botones, textos, imágenes y contenedores).
- **El rol de JavaScript:** JavaScript no dibuja directamente en la tarjeta gráfica. Lo que hace es comunicarse con la interfaz del DOM para buscar nodos, leer sus propiedades, alterar su contenido o borrar y crear etiquetas sobre la marcha. Cada vez que JavaScript cambia el DOM, el motor de renderizado recalcula el espacio de los elementos y vuelve a pintar la pantalla.

F. Mecanismos de Salida y Comunicación del Navegador

JavaScript dispone de cuatro vías principales integradas en el navegador para comunicarse con el usuario o con el desarrollador:

1. Consola de Depuración (console.log())

- **Qué es:** Es un canal directo hacia el panel de diagnóstico del navegador. No altera la página web y no es visible para el usuario común.
- **Cómo se accede:** Pulsando la tecla **F12** (o clic derecho -> *Inspeccionar*) y seleccionando la pestaña **Consola**.
- **Utilidad:** Los desarrolladores la utilizan para depurar (*debug*), verificando qué valor contiene una variable en un instante concreto o comprobando si una función se ha ejecutado.

JavaScript

```
<script>
  console.log("Salida por consola"); // Muestra el mensaje en el panel F12
</script>
```

- **Ampliación práctica:** Además del volcado simple, la consola permite categorizar trazas de error con console.error("Fallo crítico"), emitir avisos en amarillo con

console.warn("Atención") o medir tiempos exactos de cálculo con console.time("proceso") y console.timeEnd("proceso").

2. Modificación de Contenido HTML (innerHTML)

- **Qué es:** Es una propiedad que tienen los nodos del DOM para leer o sobreescribir todo el marcado y texto que contienen en su interior.
- **Mecanismo:** Primero se localiza el elemento mediante su identificador único con document.getElementById('identificador') y después se reasigna el contenido.

```
<p id="parrafito"></p>

<script>
  // Calcula la operación matemática 5 + 6 y escribe el resultado "11" dentro del párrafo
  document.getElementById("parrafito").innerHTML = 5 + 6;
</script>
```

- **Aspecto de seguridad esencial:** Si utilizamos innerHTML para meter información que ha escrito un usuario desconocido, un atacante podría escribir una etiqueta <script> maliciosa y ejecutar código en el navegador de otras personas (ataque conocido como *XSS* o *Cross-Site Scripting*). Para insertar texto plano de forma más rápida y totalmente inmune a inyecciones, se recomienda emplear la propiedad textContent.

3. Flujo Directo de Marcado (document.write())

- **Qué es:** Un método clásico que escribe texto o etiquetas HTML directamente en el flujo de la página mientras el navegador la está leyendo.

```
<script>
  document.write("<h2>Buenos días</h2>"); // Inserta directamente el encabezado en la carga
</script>
```

- **Comportamiento crítico:** Si se ejecuta document.write() mientras la página carga, funciona con normalidad. Sin embargo, si se invoca **después** de que la página haya terminado de cargar (por ejemplo, dentro de una función al pulsar un botón), el navegador **borra de forma irreversible todo el documento HTML existente** y deja únicamente lo escrito en esa llamada. Por esta razón, su uso no se recomienda en desarrollos modernos.

4. Diálogos Modales de Alerta (window.alert())

- **Qué es:** Una función que abre una pequeña ventana modal emergente nativa del sistema operativo con un mensaje y un botón de aceptar.

```
<script>
  window.alert("BUENAS NOCHES"); // Detiene la navegación hasta pulsar "Aceptar"
</script>
```

- **Mecanismo:** Pertenece al objeto global window, por lo que puede escribirse tanto window.alert() como simplemente alert().
- **Efecto bloqueante:** El diálogo es **síncrono y bloqueante**. Hasta que el usuario no pulsa con el ratón el botón "Aceptar", la ejecución del hilo principal de JavaScript se congela por completo: las animaciones se detienen y la página no atiende a ningún otro evento.

G. Capacidades Prácticas de Manipulación Dinámica

A través del DOM, JavaScript puede modificar en caliente los tres aspectos visuales de una página:

1. Modificar el Contenido de la Página Web

Permite reescribir bloques informativos al interactuar con botones u otros controles:

```
<!DOCTYPE html>
<html>
<body>
  <h1>Modificando el código HTML</h1>
  <p id="prueba">Modificando el contenido.</p>
  <!-- Al hacer clic, document.getElementById localiza el nodo y cambia su texto interno -->
  <button type="button" onclick="document.getElementById('prueba').innerHTML =
'CAMBIANDO el contenido!'">
    ¡Dale!
  </button>
</body>
</html>
```

2. Cambiar Atributos de Objetos HTML

Cualquier atributo declarado en una etiqueta HTML (como el destino href de un enlace, el ancho width o la ruta src de una imagen) se convierte en una propiedad accesible desde JavaScript.

```
<!DOCTYPE html>
<html>
<body>
  <h1>Cambio de imágenes con JavaScript</h1>
  <!-- Imagen con evento onclick que llama a la función cambiaPic() -->
  <img id="myFPImage" onclick="cambiaPic()"
src="http://myfpschool.com/wp-content/uploads/2016/06/myblack.jpeg" width="100"
height="180">
  <p>Haz click sobre las letras para cambiarlas.</p>

  <script>
  function cambiaPic() {
    var image = document.getElementById('myFPImage');
    // Con match comprobamos si el nombre del archivo contiene la palabra "green"
    if (image.src.match("green")) {
      image.src = "http://myfpschool.com/wp-content/uploads/2016/06/myblack.jpeg";
```

```
   } else {
     image.src = "http://myfpschool.com/wp-content/uploads/2016/06/mygreen.jpeg";
   }
  }
  </script>
</body>
</html>
```

- **Mecanismo:** Al hacer clic sobre la imagen, la función lee la ruta actual en image.src. Si detecta la versión en verde, conmuta el atributo hacia la imagen negra; si no, le asigna la verde. El navegador reacciona de forma automática descargando y repintando el nuevo recurso gráfico.

3. Cambiar el Estilo CSS en Tiempo Real

JavaScript puede acceder directamente a las reglas de estilo del elemento a través de la propiedad .style.

```
<!DOCTYPE html>
<html>
<body>
  <p id="mytxt">Aprende JavaScript con MyFPSchool!</p>
  <button type="button" onclick="myFunction()">¡Dale!</button>

  <script>
  function myFunction() {
    var x = document.getElementById("mytxt");
    // En CSS se escribe "font-size", pero en JavaScript se usa camelCase: "fontSize"
    x.style.fontSize = "25px";
    x.style.color = "red";
  }
  </script>
</body>
</html>
```

- **La regla sintáctica *camelCase*:** Como el signo de guion (-) representa la resta en JavaScript, las propiedades de CSS compuestas no pueden llevar guion en el código. El lenguaje sustituye el guion eliminándolo y poniendo la siguiente letra en mayúscula:
  - background-color pasa a ser style.backgroundColor
  - font-size pasa a ser style.fontSize
  - margin-top pasa a ser style.marginTop

H. El Objeto Global window y el Árbol Jerárquico del BOM

En el navegador existe una jerarquía fundamental que a menudo confunde al estudiante: **BOM (*Browser Object Model*) frente a DOM (*Document Object Model*)**.

- El objeto **window** representa la ventana o pestaña completa del navegador y es el **objeto raíz global** del entorno cliente.
- El **document** (el DOM) es en realidad una propiedad que cuelga directamente de window (window.document).
- **Regla de ámbito global:** En JavaScript para navegadores, cualquier variable o función declarada a nivel superior (con var) o cualquier método nativo de la ventana pasa a formar parte de window. Por esa razón exacta, las dos siguientes líneas son técnica y funcionalmente idénticas:

```
window.alert("Mensaje"); // Invocación formal completa
alert("Mensaje");        // Invocación simplificada aprovechando el ámbito global
```

I. Profundización en el Renderizado: *Reflow* (Layout) y *Repaint*

Cuando un script altera el DOM o los estilos, el navegador no responde de forma mágica; ejecuta una cadena de operaciones costosas en recursos de la CPU y la tarjeta gráfica:

1. **Reflow (o Re-layout):** Ocurre cuando un cambio de JavaScript altera las **dimensiones geométricas o la posición** de un elemento (por ejemplo, al modificar x.style.fontSize = "25px" o al inyectar bloques con innerHTML). El navegador debe recalcular el espacio que ocupa ese nodo y cómo desplaza a todos los elementos circundantes en la página.
2. **Repaint:** Ocurre cuando se modifica una propiedad meramente visual que **no altera el espacio físico** (por ejemplo, cambiar el color del texto con x.style.color = "red" o el fondo con backgroundColor). El navegador no recalcula posiciones, solo repinta los píxeles afectados.

- **Lección para el programador:** Los cambios que provocan *reflow* continuos dentro de un bucle ralentizan la página web. Las modificaciones visuales deben agruparse para evitar parpadeos y caídas en los fotogramas por segundo (FPS).

J. Actividades Prácticas y de Consolidación

- **Ejercicio 1:** Añadir un segundo botón al ejemplo de los apuntes, al pulsarlo, no solo altere el párrafo sino que cambie también el texto del encabezado <h1> mediante document.getElementById() y innerHTML.

```
<!DOCTYPE html>
<html>
<body>
  <h1>Modificando el código HTML</h1>
  <p id="prueba">Modificando el contenido.</p>
  <!-- Al hacer clic, document.getElementById localiza el nodo y cambia su texto interno -->
```

```
  <button type="button" onclick="document.getElementById('prueba').innerHTML =
'CAMBIANDO el contenido!'">
    ¡Dale!
  </button>
</body>
</html>
```

- **Ejercicio 2:** Implementar una página que capture el evento clic de un botón para emitir una traza informativa a las herramientas de desarrollo mediante console.log().
- **Ejercicio 3:** Modificar el ejercicio anterior sustituyendo la escritura en el párrafo por ventanas de aviso emergentes modales utilizando el método window.alert() / alert().
- **Ejercicio 4:** Construir una web con tres botones ("Ruso", "Español", "Inglés") que alteren un párrafo ( ) mostrando un saludo en el idioma elegido y aplicando a dicho párrafo un color de fuente CSS diferente para cada uno mediante .style.color.
- **Ejercicio 5:** Adaptar el mismo ejercicio para que las salidas en los tres idiomas se impriman únicamente a través de la consola de desarrollador.
- **Ejercicio 6:** Transformar el código para generar los textos directamente en el flujo de la página mediante el método document.write().
- **Ejercicio 7.** Diseña una interfaz compuesta por un titular <h1>, un párrafo Sistema en espera y tres botones:
    1. *Botón 1 (Consola):* Emite una traza mediante console.log() indicando la hora del sistema.
    2. *Botón 2 (Estilo):* Modifica el color de fondo del párrafo a verde y su texto a "Sistema Activo" mediante innerHTML y .style.backgroundColor.
    3. *Botón 3 (Alerta):* Lanza un cuadro modal mediante window.alert() avisando de que el proceso ha concluido.
- **Ejercicio 8:** Desarrollar un test interactivo de siete preguntas con botones «Verdadero» y «Falso». Cuando el usuario hace clic, el script evalúa el acierto y modifica la propiedad de estilo .style.color = "green" (acierto) o .style.color = "red" (error).
- **Ejercicio 9:** Crear una secuencia cíclica de al menos 4 imágenes fotograma a fotograma. Al hacer clic sobre la imagen, el script comprueba cuál se está visualizando (mediante una condición o extrayendo un contador de un array) y reasigna el atributo .src para mostrar la siguiente.